

1.Hello World Container

Step1- Create Dockerfile

```
# Use a minimal base image
FROM alpine:latest
# Define the command to print "Hello, World!"
CMD ["echo", "Hello, World!"]
```

Step2.Build the Docker Image

```
docker build -t hello-world .
```

Step3.Run the Docker Container

```
docker run hello-world
```

2.Flask application on an EC2 instance using Docker.

Flask App Deployment on EC2 using Docker provides a step-by-step guide to deploying a Flask application on an EC2 instance using Docker.

Prerequisite Basic knowledge of Flask, Docker, and AWS EC2.

Project Structure

Step1 .Git clone from github repo

```
flask-app-ecs/
├── Dockerfile
├── README.md
├── app.py
├── requirements.txt
└── run.py
```

step2 . ## Dockerfile

The `Dockerfile` is used to build a Docker image for the Flask application.

1

```
``Dockerfile
```

```
# Base image (OS)
```

```
FROM python:3.9-slim
```

```
# Working directory
```

```
WORKDIR /app
```

```
# Copy src code to container
```

```
COPY . .
```

```
# Run the build commands
```

```
RUN pip install -r requirements.txt
```

```
# Expose port 80
```

```
EXPOSE 80
```

```
# Serve the app / run the app (keep it running)
```

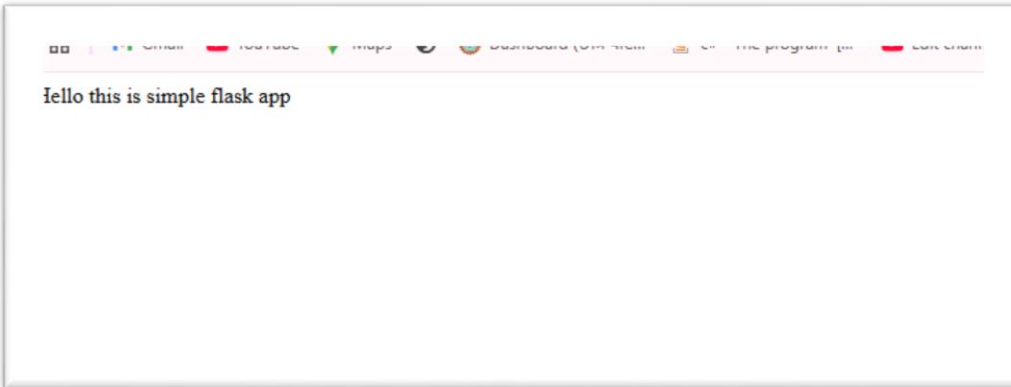
```
CMD ["python", "run.py"]
```

Step3. Build image

```
Docker build -t flask-app .
```

Step4. Run Container

```
Docker run -d -p 80:80 flask-app
```



3. Node.js Express App in Docker

Project Structure

Step1. Git clone from github repo

Node app

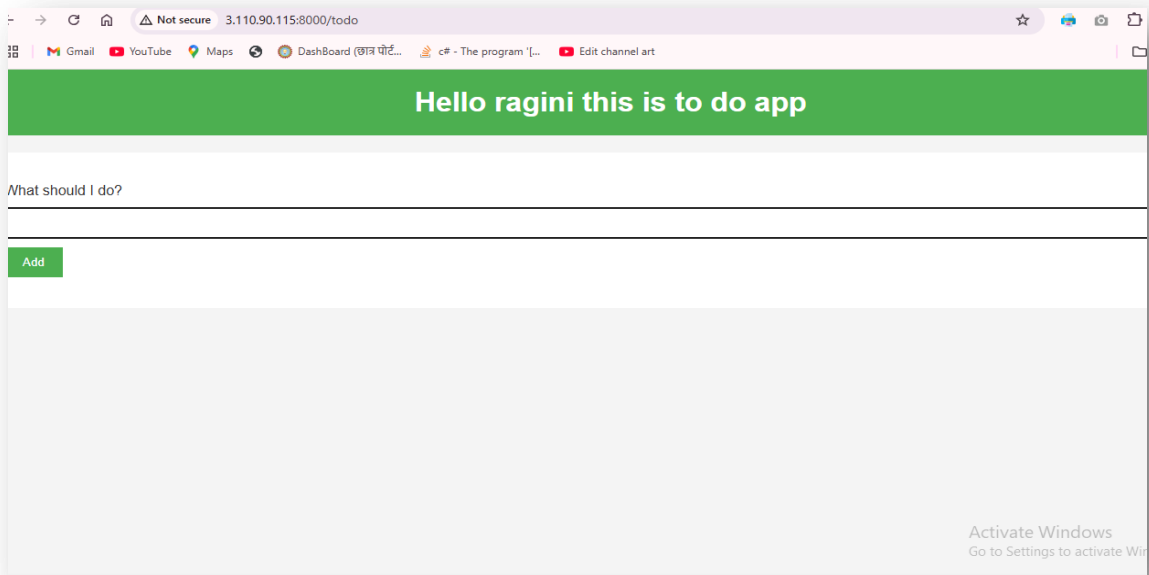
```
|— Dockerfile
|— README.md
|— docker-compose.yaml
|— package-lock.json
|— package.json
|— test.js
└— app.js
```

Step2. Build the Docker image:

`docker build -t node-app .`

Step3. Run the Docker container:

`docker run -d -p 8000:8000 nodejs-app`



4.MySQL Database in Docker

Step 1 Pull mysql images form docker hub

Docker pull mysql

Step 2 To run images

```
docker run -e MYSQL_ROOT_PASSWORD=root -d mysql
```

Step3.connect mysql from Inside the Container

```
docker exec -it <container ID> bash
```

step4. inside bash run

```
mysql -u root -p
Enter password:
```

step5. Persist Data with Volumes

By default, Docker containers are ephemeral, meaning data will be lost if the container is removed. To persist MySQL data, use Docker volumes.

Docker volume create mysql-data

```
docker run -d --name mysql -v mysql-data:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=root
```

```
mysql:latest
```

5. PostgreSQL with Adminer

- Create a `docker-compose.yml` file.
- Run `docker-compose up -d`.
- Access Adminer at <http://localhost:8080>.

```
docker-compose.yml
```

```
version: '3.8'
```

```
services:
```

```
  postgres:
```

```
    image: postgres:latest
```

```
    container_name: postgres_db
```

```
    restart: always
```

```
    environment:
```

```
      POSTGRES_USER: ragini
```

```
      POSTGRES_PASSWORD: root
```

```
      POSTGRES_DB: mydata
```

```
  volumes:
```

```
    - postgres_data:/var/lib/postgresql/data
```

```
  ports:
```

1

- "5432:5432"

adminer:

image: adminer

container_name: adminer

restart: always

ports:

- "8080:8080"

depends_on:

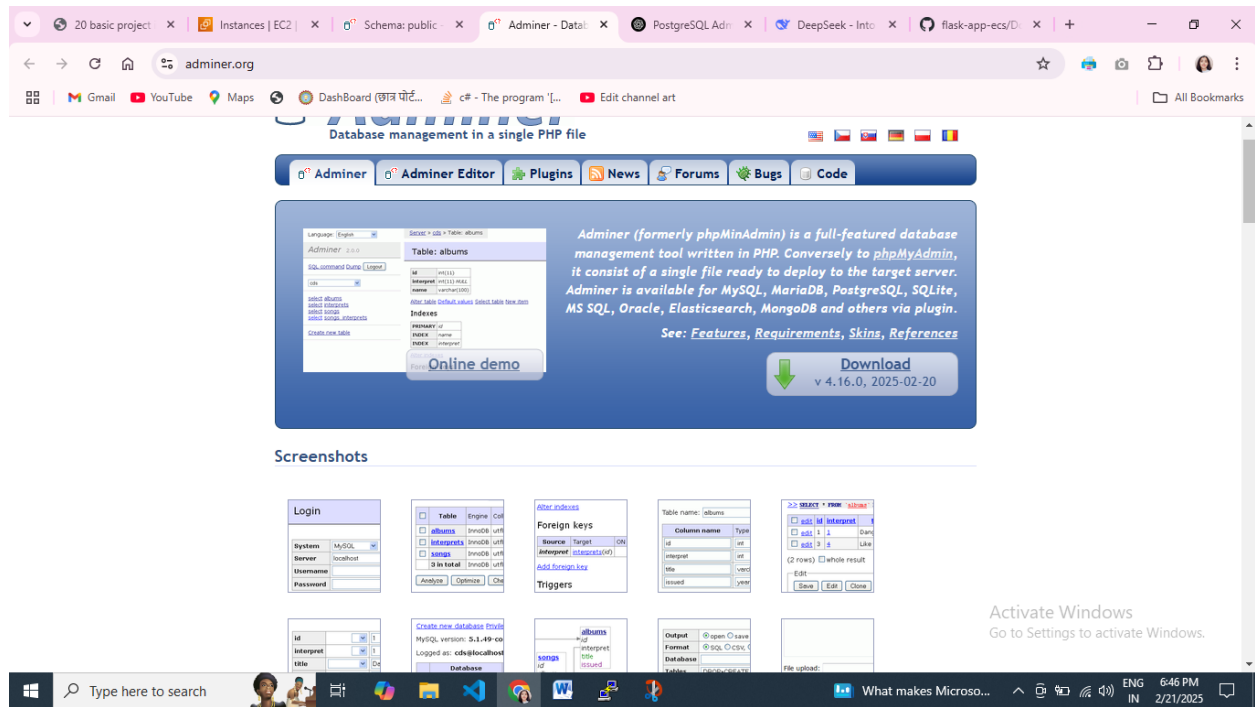
- postgres

volumes:

postgres_data:

postgres *Adminer* 4.8.1 [4.16.0](#)

DB: 



Postgress sql



6.WordPress with MySQL

Mkdir wordpress

Step 1: Create a `docker-compose.yml` File

```
version: '3.8'
```

```
services:
```

```
  db:
```

```
    image: mysql:5.7
```

```
    container_name: wordpress_db
```

```
    restart: always
```

```
    environment:
```

```
      MYSQL_ROOT_PASSWORD: root
```

```
      MYSQL_DATABASE: wordpress
```

```
      MYSQL_USER: ragini
```

```
      MYSQL_PASSWORD: root
```

```
    volumes:
```

```
      - db_data:/var/lib/mysql
```

```
  wordpress:
```

```
    image: wordpress:latest
```

```
    container_name: wordpress_app
```

```
    restart: always
```

```
    ports:
```

```
      - "8000:80"
```


1

environment:

WORDPRESS_DB_HOST: db:3306

WORDPRESS_DB_USER: ragini

WORDPRESS_DB_PASSWORD: root

WORDPRESS_DB_NAME: wordpress

volumes:

- wordpress_data:/var/www/html

depends_on:

- db

volumes:

db_data:

wordpress_data:

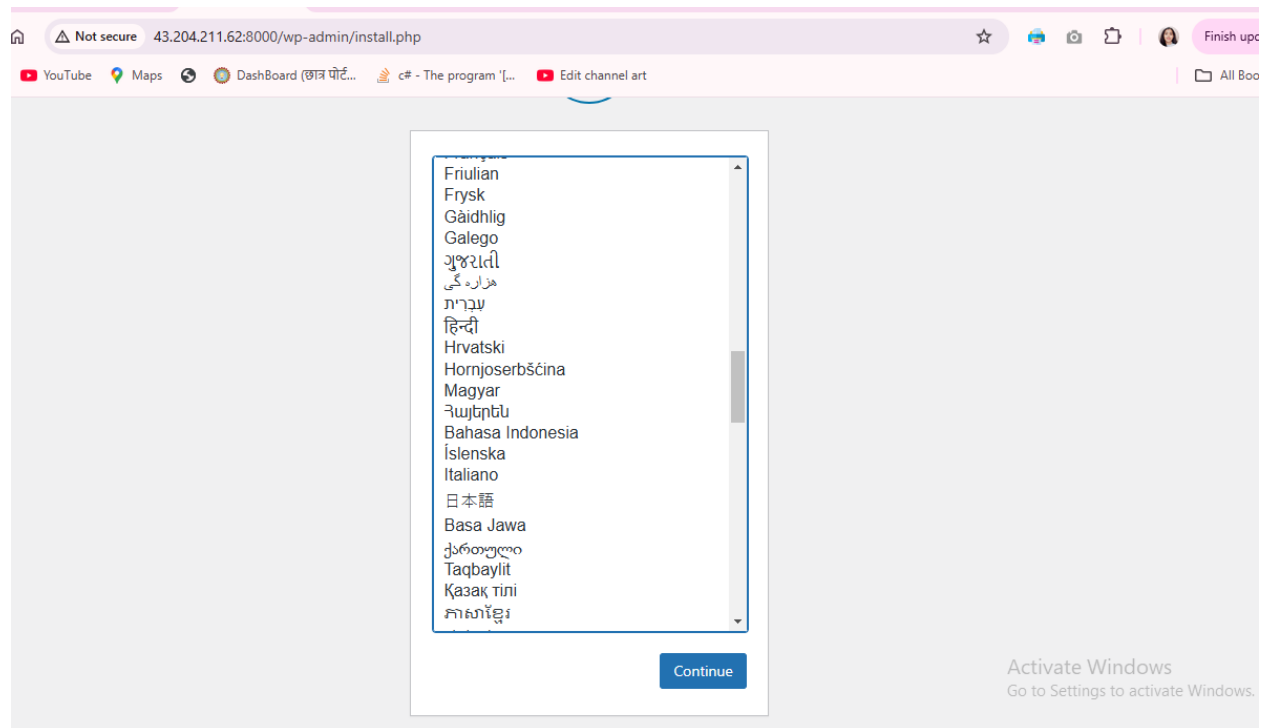
Step 2: Start the Containers

`docker-compose up -d`

check running container

🔗 Open your browser and go to:

<http://<ip>:8000>



```
docker logs wordpress_db
```

If everything is fine, you should see messages like:

docker exec -it wordpress_db bash:

```
bash
```

```
mysql -u root -p
```

```
ubuntu@ip-172-31-13-29: ~/projects/wordpress
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> ^C
mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
| wordpress |
| wpdata |
+-----+
6 rows in set (0.01 sec)

mysql> ^C
mysql> ^C
mysql> use wordpress;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> show tables;
+-----+
| Tables_in_wordpress |
+-----+
| wp_commentmeta |
| wp_comments |
| wp_links |
| wp_options |
| wp_postmeta |
| wp_posts |
| wp_term_relationships |
| wp_term_taxonomy |
| wp_termmeta |
| wp_terms |
| wp_usermeta |
| wp_users |
+-----+
12 rows in set (0.00 sec)

mysql>
```

Activate Windows
Go to Settings to activate Windows.

Type here to search

29°C Mostly clear

ENG IN 7:05 PM 2/21/2025